

Indian Stock Data Hub

A free, open-source research desk for Indian listed companies. Enter a few ticker symbols and get each company's key numbers, quarterly trends, latest filings, recent news and a plain-English analysis, then export it all to Excel and Word. The optional AI runs entirely on your own computer.

1 What the app does

2 Before you start

3 Install the app

4 Choose a model

5 Your first analysis

6 Reading the results

7 Analysing a whole watchlist

8 All the numbers, in one place

9 What changed since last time

10 What management promised

11 Your portfolio, and your trades

12 Screening what you have analysed

13 Comparing companies side by side

14 Searching everything you have downloaded

15 Ask AI: answers from your own data

16 Excel and Word reports

17 Finding ticker symbols

18 Privacy and how the AI is used

19 The Settings page

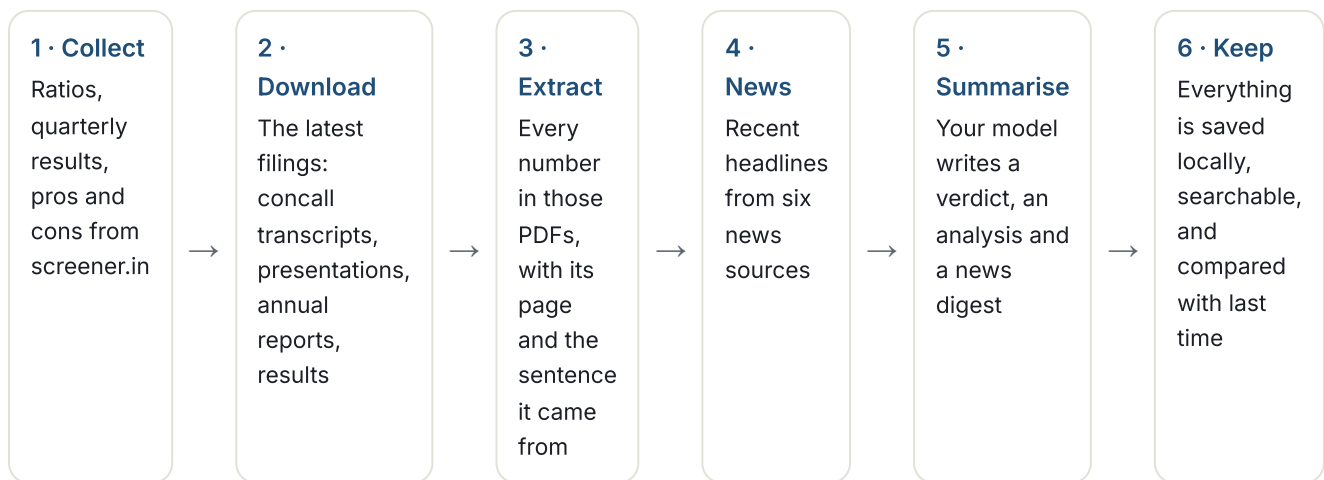
20 Keeping it light, and starting it automatically

21 Troubleshooting

22 Glossary

What the app does

You type in company tickers; the app collects public information about each one, organises it, and optionally asks a local AI model to summarise it.



What you get for each company

- Key ratios: market cap, price, P/E, ROE, ROCE and more
- 13 quarters of results, with trend charts
- Strengths and risks
- An AI verdict (Positive, Neutral or Cautious) with a short analysis
- Recent news with an AI digest and sentiment
- The downloaded filings, and every figure found inside them
- A detailed table lining up every metric against every source that reports it
- A list of what has changed since the last time you looked
- What management promised on the earnings call, and whether it happened
- A formatted Excel workbook and a Word report

What it isn't

- **Not investment advice.** The AI can be wrong. Use it as a starting point for your own research.
- **Not real-time.** Prices and ratios are as shown on screener.in when you run the analysis.
- **Not a trading tool.** It reads public information; it never connects to a broker.

The AI part is optional. Without it you still get all the data, charts, news headlines, the detailed tables, the archive search and the reports.

Before you start

Here's what your computer needs. Setting everything up takes 15 to 30 minutes the first time, mostly waiting for downloads.

You need	Details
A computer	Windows 10 or 11, macOS, or Linux
Python 3.10 or newer	The programming language the app is written in. Free from python.org/downloads . Chapter 3 shows how to check whether you already have it.
Internet connection	To fetch company data, filings and news. The AI itself works offline.
Disk space	About 500 MB for the app and its libraries, plus room for filings (roughly 10 to 40 MB per company). The AI model is extra: 0.4 to 8 GB.
Memory (RAM)	4 GB is enough without AI. For the recommended AI model, 16 GB or more.
A web browser	Chrome, Edge, Firefox or Safari. The app opens in your browser but runs on your computer.

New to the terminal? Some steps ask you to type commands. On Windows, open **PowerShell** (search for it in the Start menu). On macOS, open **Terminal** (press `⌘` `Space` and type "Terminal"). Type or paste each command, then press `Enter`. Lines starting with `#` in this guide are comments; you don't type them.

Install the app

Five steps, done once. After that, starting the app is a single command (step 5).

1 Check for Python, install it if needed

WINDOWS

```
python --version
```

If you see an error or a version below 3.10, download Python from python.org. **On the first installer screen, tick "Add python.exe to PATH"**. Then close and reopen PowerShell.

MACOS / LINUX

```
python3 --version
```

If it's missing or older than 3.10, install it from python.org (macOS), with Homebrew (`brew install python`), or with your Linux package manager.

2 Download the app

Easiest: on the project's GitHub page, click the green **Code** button, then **Download ZIP**. Unzip it somewhere easy to find, such as your Documents folder.

With git (if you have it):

```
git clone https://github.com/tapariapurv/indian-stock-data-hub.git
```

Then move into the project folder. Replace the path with wherever you put it:

```
cd Documents/indian-stock-data-hub
```

3 Create a private Python environment

This keeps the app's libraries separate from anything else on your computer. You do this once, inside the project folder.

WINDOWS

```
python -m venv .venv
.venv\Scripts\activate
```

MACOS / LINUX

```
python3 -m venv .venv
source .venv/bin/activate
```

Your prompt now starts with `(.venv)` . That means the environment is active.

Windows says "running scripts is disabled"? Skip the activate line and put `.venv\Scripts\` in front of the commands in steps 4 and 5, for example `.venv\Scripts\python -m pip install -r requirements.txt` .

4 Install the app's libraries

```
python -m pip install -r requirements.txt
```

This downloads Streamlit (the app framework), the PDF reader, and the Excel and Word writers. It takes a minute or two.

5

Start the app

```
python -m streamlit run app.py
```

Your browser opens at <http://localhost:8501>. If it doesn't, open that address yourself. The first time, Streamlit may ask for an email address; just press `Enter` to skip.

To stop the app, click the terminal window and press `Ctrl C`. **To start it again another day**, open a terminal in the project folder, activate the environment (step 3's second line), and run step 5.

Choose a model

The app works without any model at all. If you want the verdicts, the written analysis, the news digests, the transcript reading and the archive answers, you need to point it at one — and there are four quite different ways to do that. Pick whichever suits your machine, your budget and your privacy needs. You can change your mind at any time from **Settings** → **Models & keys**; nothing else in the app changes.

Route	Good when	Cost	Your data
A · Ollama	You want the simplest local setup. This is the default.	Free	Never leaves the machine
B · A local app	You already use LM Studio, Jan, llama.cpp or similar.	Free	Never leaves the machine
C · A hosted API	You want the strongest models and don't mind paying.	You pay per token	Sent to that provider
D · Your own server	You have a GPU box, a work server or a rented instance.	Whatever it costs you	Stays on that server
E · No model	You only want the data, tables, charts and reports.	Free	Nothing to send

Route A · Ollama on this computer

1

Install it

Download from ollama.com/download. On Windows and macOS it is an ordinary installer; on Linux run `curl -fsSL https://ollama.com/install.sh | sh`. Version 0.9 or newer is needed.

2

Check it is running

Open <http://localhost:11434> in a browser. It should say *Ollama is running*. On Windows and macOS it starts with the computer; on Linux you may need `ollama serve`.

3

Download a model that fits your machine

Browse ollama.com/library and pull one with `ollama pull <name>`. Any chat model works. The thing that matters is size: a model needs to fit in memory alongside everything else you are running, and a rough guide is that the download size is close to the memory it will want.

Your RAM	Aim for	What to expect
8 GB	A model of about 1–2 GB	Basic Fast, but small models misread financial context — one we tested called a 7.6% return on equity a "strong position". Treat the verdicts with real caution.
16 GB	A model of about 4–8 GB	The sweet spot Accurate summaries that quote the right numbers, typically a second or three each.
24 GB or more	A model of 8 GB and up	Better written analysis and noticeably better at reading transcripts. Slower per answer.

Names and versions change constantly, so this guide deliberately doesn't recommend one. Sort the library by popularity and take a recent general-purpose chat model in your size bracket. Avoid anything labelled *embedding* — the app hides those, because they cannot write.

4 Point the app at it

Go to **Settings** → **Models & keys**, choose *Ollama (local)*, and press **Test connection**. Every model you have pulled appears in the list with its size. Pick one and save. The research sidebar also has a model dropdown, so you can switch between your installed models for a single run without changing the saved setting.

Route B • An app you already run locally

LM Studio, Jan, llama.cpp's server, text-generation-webui and most others can expose an OpenAI-compatible endpoint. Turn that server on in the app's own settings and note the address it prints — commonly something like `http://localhost:1234/v1`. Then in **Settings** → **Models & keys** choose *Custom / self-hosted*, paste that address, leave the key empty, and press **Test connection**. The models that app has loaded will appear in the list.

Route C • A hosted API

Anthropic, OpenAI, Google and OpenRouter are supported directly. OpenRouter is worth knowing about: one key gives you access to models from many companies, which is handy if you want to try several without opening several accounts.

Provider	Get a key from	Environment variable
Anthropic (Claude)	console.anthropic.com	ANTHROPIC_API_KEY
OpenAI	platform.openai.com	OPENAI_API_KEY
Google (Gemini)	aistudio.google.com	GOOGLE_API_KEY
OpenRouter	openrouter.ai	OPENROUTER_API_KEY

Sign up, create an API key, then paste it into **Settings** → **Models & keys** and press **Test connection**; the provider's models load into the dropdown. Two notes on keys. They are saved in `settings.json` beside the app, readable only by your user account and never committed to git — but if you would rather not store one at all, set the environment variable above instead and leave the box empty,

because the app always prefers the variable. And a hosted model means the scraped ratios, headlines and retrieved passages are sent to that company; nothing else is.

Route D • A server somewhere else

A faster machine on your network, a work server, or a GPU instance you rent by the hour. If it runs Ollama, choose *Ollama (local)* and replace the address with that machine's, for example `http://192.168.1.20:11434`. If it speaks the OpenAI API, choose *Custom / self-hosted* and give it the address and a key if it needs one. Either way the app behaves exactly as it does locally.

Remember that the remote machine must be reachable from this one, and that Ollama only listens to other computers if it was started with `OLLAMA_HOST=0.0.0.0` on that machine.

Route E • No model at all

Turn off **Use a model** in Settings, or just never set one up. You keep every ratio, every quarterly table, every extracted figure, the correlated numbers, the news headlines, the archive keyword search, the saved history and both reports. What you lose is the written verdict, the analysis paragraph, the news digest, transcript reading and the archive's ability to reason about what it finds.

Keeping a paid model on a leash

If you choose a hosted API, set your limits before your first big run. Open **Settings** → **Spending** and you will find three:

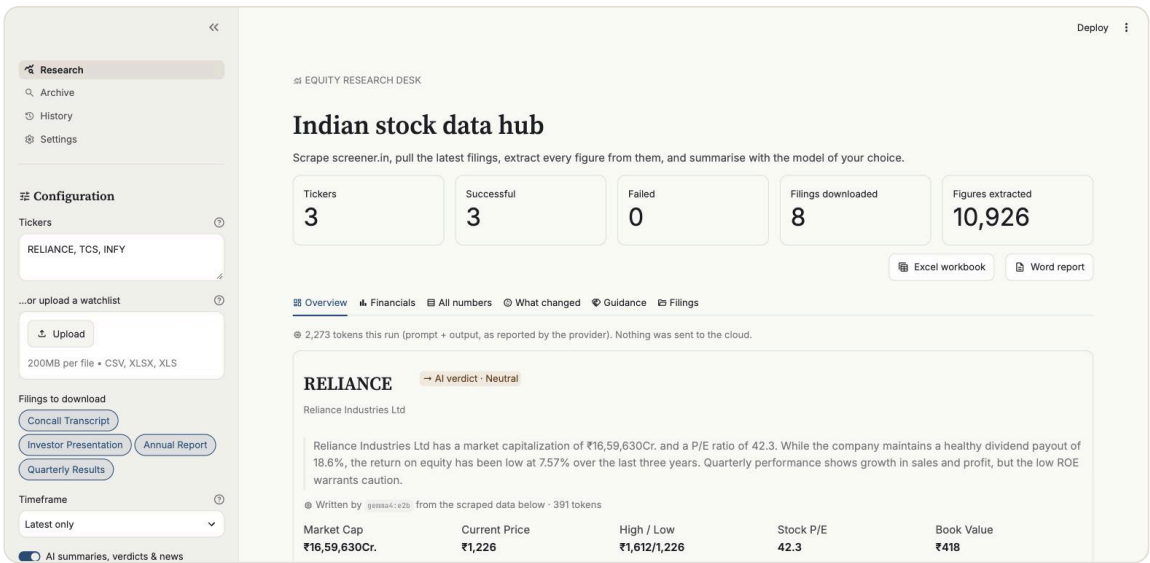
Limit	What it does
Tokens per run	The one that actually saves you. A fifty-company watchlist against an expensive model can spend a month's budget in a single click; this stops the run instead.
Monthly limit	A ceiling in your own currency, counted from the first of the month.
Monthly token limit	The same idea counted in tokens, useful when you are on a free tier with an allowance.

A call that would cross a limit is refused before it is sent, so a limit cannot be passed by accident; the app tells you plainly that it stopped, and everything else in the run carries on as normal. For the money figures to mean anything, enter what your provider charges per million tokens — input and output — next to its key. The app does not ship a price list, because prices change and a stale one would be worse than none. Local models are always free, are recorded as free, and are never blocked by a limit.

For scale: a company review is about 400 tokens, a news digest about 300, and an archive question about 2,800 including the search steps. Answers are cached, so re-running the same company costs nothing at all.

Your first analysis

Everything is controlled from the sidebar on the left. The defaults work well, so you can simply press **Run analysis**.



The app after analysing three companies: sidebar on the left, summary tiles and export buttons at the top, one card per company below.

Sidebar control	What it does
Tickers	The companies to analyse, separated by commas, e.g. RELIANCE, TCS, INFY . Use screener.in symbols; chapter 17 shows how to find them. Start with 1 to 5 companies.
Filings to download	Click to turn each filing type on or off: concall transcripts, investor presentations, annual reports and quarterly results. Fewer types means a faster run.
Timeframe	How many of each filing to fetch: latest only (fastest), the last 2 or 4, or all available.
Local AI switch	Turns the AI verdicts, analyses and news digests on or off. Greyed out if Ollama isn't running.
Model	Which model to use for this run, from whichever provider you set up in chapter 4. Changing it here applies to this run only.
Run analysis	Starts the run. Changes to the controls above only apply when you press it.
User guide (PDF)	Opens this guide.

How long does it take? The first run for a company takes 20 to 60 seconds, mostly downloading and reading its PDFs. A progress bar shows which company is being processed. Results are remembered for an hour, so re-running is almost instant, and downloaded PDFs are kept in the `downloads` folder so they are never fetched twice.

Reading the results

Results are organised into three tabs: **Overview**, **Financials** and **Filings**.

Overview: one card per company

The screenshot shows a company card for Reliance Industries Ltd. The card is divided into several sections:

- AI Analysis:** A paragraph summarizing the company's market capitalization, P/E ratio, dividend payout, and quarterly performance.
- Key Ratios:** A table showing Market Cap, Current Price, High / Low, Stock P/E, Book Value, Dividend Yield, ROCE, and ROE.
- Strengths and Risks:** Two sections listing key points. Strengths include a healthy dividend payout. Risks include stock trading at a premium to book value and low return on equity.
- Recent News:** A list of headlines with a short AI digest and overall sentiment (Mixed).
- Filings:** A section for downloading documents like the Transcript, Investor Presentation, Annual Report, and Quarterly Results.

A company card: the AI analysis, key ratios, strengths and risks, and recent news with its AI digest.

- **AI verdict** (green Positive, amber Neutral, red Cautious) and a short analysis. The line under it names the model that wrote it and how many tokens it used.
- **Key ratios** as shown on screener.in. See the glossary for what each one means.
- **Strengths and risks:** screener.in's automatic pros and cons.
- **Recent news:** up to 8 headlines from the last 14 days, a short AI digest, and an overall sentiment (Positive, Mixed or Negative). Click a headline to read the article.
- **Figures extracted from filings:** click to open a searchable table of every number found in the downloaded PDFs, with its page and the sentence it came from. These are found by pattern matching, so check the context before relying on one.

Where the verdict comes from

The one-word verdict is *not* the model's opinion. It is worked out by the app, from the ratios, using three rules applied in order:

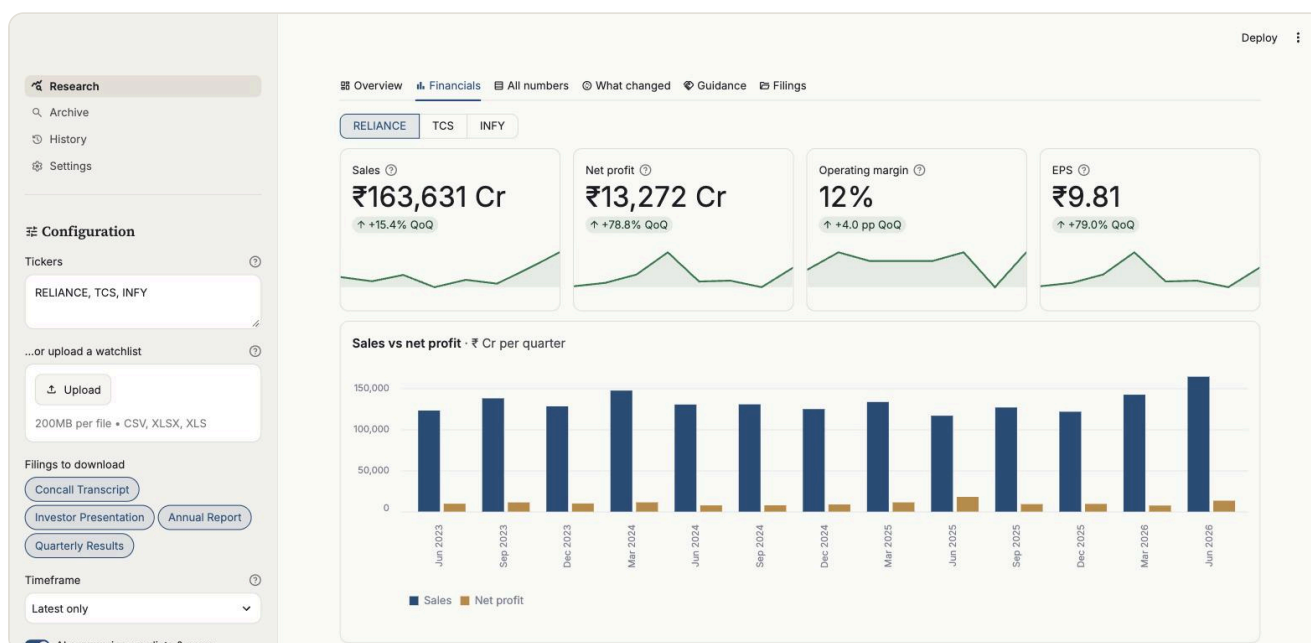
Verdict	When
Cautious	Return on equity below 12%, <i>or</i> debt to equity above 1.5, <i>or</i> net profit fell between the two quarters shown.
Positive	Return on equity above 18% <i>and</i> net profit rose between the two quarters shown.
Neutral	Everything else.

The model is told the verdict and asked only to explain it in plain English, quoting the figures behind it. This is deliberate: small local models are poor at comparing numbers — one we tested called a 7.6% return on equity a "strong position" every single time — but good at writing a sentence about figures they have been handed. Doing the arithmetic in code makes the badge identical whichever model you use, and costs nothing.

Where the ratios needed are missing, there is no rule to apply and the model is asked for the verdict after all.

They are rules of thumb, not a valuation. A good company can be Cautious for a quarter, and a weak one Neutral. They exist so the badge means the same thing every time — read the analysis and the numbers, not just the colour.

Financials: trends at a glance



The Financials tab: latest-quarter figures with their change from the previous quarter, a sales-versus-profit chart, and the full results table.

Pick a company at the top. The four cards show the latest quarter's sales, net profit, operating margin and EPS. The small arrow shows the change from the previous quarter; the line underneath shows the last 8 quarters. Below are a chart of 13 quarters and the complete table, which you can download as CSV.

The other tabs

All numbers lines every metric up against every source that reports it, and lets you search all the extracted figures. **What changed** compares this run with the last saved one. **Guidance** tracks what management promised on the earnings call. Each has a chapter of its own later in this guide.

Filings: the source documents

For each company: which filings were downloaded (or why one was skipped, for example because it was larger than 25 MB or wasn't a PDF), with links to the originals. Expand *All document links* to see every document screener.in lists for the company.

Analysing a whole watchlist

Typing tickers one at a time is fine for three companies. For thirty, hand the app a file.

The file it expects

A CSV or Excel file with a column headed `ticker`. Anything else in the file is ignored, so your existing watchlist export will usually work as it is. If there is no column called `ticker`, the app also accepts `symbol`, `scrip`, `code` or `NSE code`, and failing all of those it reads the first column.

ticker	notes
RELIANCE	core holding
TCS	
INFY	watching

The sidebar has a **Watchlist template (CSV)** button that downloads exactly this file, already filled in with three examples. Open it in Excel, replace the rows, save, and upload it.

Running it

Click **Upload** in the sidebar, choose your file, and press **Run analysis**. The uploaded list replaces whatever is in the Tickers box for that run. Duplicates are removed and anything that isn't a plausible symbol is skipped, so a stray heading row or a blank line does no harm.

How long it takes

Budget roughly five to fifteen seconds per company on the first run, depending on how many filings you asked for and how fast your model is. The app pauses briefly between companies on purpose — the sites it reads are free and public, and hammering them is both rude and a good way to get blocked. You can shorten or lengthen that pause in **Settings** → **Data & scraping**.

Second and later runs are far quicker. A filing that has already been read is never read again, and an analysis of unchanged data comes straight back from the cache at no cost — the token counter will say *0 tokens this run*.

All the numbers, in one place

The summary tells you the story. The **All numbers** tab shows you every number behind it, with each source of that number lined up beside the others.

What the table shows

One row per metric. Reading across, you get the value screener.in publishes, what the last two quarters reported and the change between them, then how the downloaded filings describe the same metric: how many times it appears, the most recent value, and the lowest, middle and highest of every mention.

Column	What it means
Screener ratio	The figure shown on the company's screener.in page.
Latest / Previous quarter	From the quarterly results table, in ₹ crore or per cent.
QoQ	Change from the previous quarter to the latest one.
In filings	How many times this metric was found across the PDFs.
Filing latest	The first mention in the most recent document.
Filing low / median / high	The spread across every mention.
Sources	Which kinds of filing the number came from.

Why the spread matters

A wide gap between *low* and *high* is a warning, not an error. It usually means the label caught more than one kind of number — a figure in crore on one page and the same metric as a percentage on another, or a table header that lined up with the wrong column. When something looks odd, use the figures table underneath: it shows every extracted number with its page and the exact sentence it came from, so you can check the source in seconds.

Finding a number

Recognised metrics only is on by default, and limits the table to metrics the app could identify. Turn it off to see every label the extractor produced, including the rough ones. Below, the figures table can be filtered by filing type and category, or searched by text — typing `margin` finds every mention of margin across every downloaded document for that company.

Both tables download as CSV, and the whole thing also arrives as an **All numbers** sheet in the Excel workbook and a cross-checked table in the Word report.

What changed since last time

Every analysis is saved. The next time you run the same company, the app tells you what moved.

The What changed tab

After a run, this tab compares each company with the last saved analysis of it and lists the differences: ratios that moved, strengths and risks that appeared or disappeared, filings that are new since you last looked, headlines you haven't seen, and any change in the AI's verdict. The first time you analyse a company there is nothing to compare, so the tab says so; run it again next week and it fills up.

This comparison is plain arithmetic and text matching. No model is involved, deliberately — a "what changed" list that could invent a change would be worse than no list at all.

The History page

The **History** page in the sidebar holds every saved analysis. Pick a company and a date to reopen exactly what the app showed you then: the verdict, the ratios, the strengths and risks, the news and the full correlated table. Above it, a chart plots any ratio across all your saved runs, so you can watch the P/E or the ROE drift over months without keeping a spreadsheet.

Each saved analysis also shows what changed between it and the run before it, so you can step back through a company's history one analysis at a time.

Getting back to the paper

A saved analysis is only worth having if you can trace it. Under **Where this came from**, every filing behind that run is listed with the file it was saved as, the full path to it on your disk, the span of pages its figures came from, and a link to the original document on the exchange or company website. In the numbers table, each metric carries a **Pages** column listing the exact pages that figure appeared on — so checking a number means opening the PDF and turning to the page, not hunting through sixty of them.

What management promised

Every quarter, management says things on the earnings call: margins will improve, growth will pick up, capex will land at a number. Nobody ever checks. This does.

Reading the transcript

Open the **Guidance** tab and press **Read the latest transcript**. The app finds the sentences where management commits to a future outcome and files each one as a metric, a commitment and a deadline. You need a concall transcript downloaded for that company, so tick **Concall Transcript** in the sidebar before the run.

This is extraction, not judgement, which is exactly what small local models are good at. The app filters the transcript down to the couple of dozen candidate sentences first, so reading a sixty-page transcript costs a few hundred tokens rather than tens of thousands.

Checking it later

Once a later quarter has been reported, press **Check against results**. Each commitment from an earlier quarter is graded **Delivered**, **Missed** or **Unclear** against the numbers the app has actually scraped — never against anything the model remembers. The **Delivered** counter at the top is the useful number: it tells you, over time, how often this management does what it says it will.

Treat a single grade as a prompt to go and read the transcript yourself, not as a verdict. Treat a pattern across eight quarters as worth knowing.

Your portfolio, and your trades

The Research page answers "what is this company like?". The Portfolio page answers "what do I own, what is it worth, and how have I actually done?" — and it keeps working in the background while you get on with something else.

Accounts and holdings

A portfolio is made of **accounts** — call them what you like: *Long term*, *Trading*, *Mum's*. Search any NSE or BSE listed company and add it to one. Quantity and buy price are optional: leave them blank and the app is a watchlist; fill them in and it shows value, profit and loss, and how much of your money sits in any one stock or sector.

Adding a stock queues its first full analysis in the background — the same scrape, filings and verdict the Research page produces. You do not have to wait for it.

Nothing leaves your computer. Accounts, quantities and buy prices live in `~/stock-data-hub`, outside the app folder, so updating the app never touches them. Prices are looked up by ticker symbol alone.

Trades: what you actually bought and sold

A quantity and an average price are a snapshot. They cannot tell you what you made on the shares you already sold, or what your money actually earned. The **Trades** section fixes that: record each buy and sell — date, quantity, price, and any brokerage — and the app works the rest out.

- **What you still hold** and its average cost, recalculated from your trades. You no longer type these in; they are derived.
- **Realised gains**, matched **first-in first-out** — the oldest shares are treated as sold first, which is the basis Indian equity taxation uses. Charges are counted inside the cost of a buy and taken off the proceeds of a sell, so a round trip eaten entirely by brokerage correctly shows as no profit.
- **Short term or long term**, split at the twelve-month line, and totalled per financial year (April to March).
- **A money-weighted return (XIRR)** — what your money earned given *when* you put it in and took it out. Two people can own the same stock and earn very different returns depending on when they bought; this is the number that shows it.

This is a record of your trades, not tax advice. The app splits short from long term and totals each, but deliberately does not calculate tax owed: rates, surcharges and exemptions change, and a stale rate built into an app is worse than none at all. Take the figures to whoever does your return.

A stock with no trades recorded keeps whatever quantity and price you typed in, so nothing changes until you choose to use this. If you sell more shares than the app has purchases for, it says so rather than inventing a cost.

Allocation, alerts and the weekly digest

Allocation shows where your money actually sits, by sector and by company size, and warns when one stock or one sector grows past the limits you set in Settings. **Alerts** collects changed verdicts, new filings, missed guidance and your own price alerts into an inbox, and can send them as a notification or an email. **Digest** is the weekly summary of all of it.

The Stock page

Clicking any holding opens its own page: a candlestick chart with 50- and 200-day averages, upcoming results and dividend dates, every analysis ever saved for it, and one line that is hard to get anywhere else — **today's P/E against the company's own five-year range**.

A P/E on its own says very little; 43 is cheap for one business and dear for another. Against its own history it starts to mean something: "43, and the 42nd percentile of its own five-year range" says this company has usually been more expensive than it is today. The app shows the low, median and high alongside it.

Read it as a question, not an answer. It compares a company only with its own past, never with its peers, and a company can be re-rated for perfectly good reasons — or stay cheap for perfectly bad ones. A loss-making period has no meaningful P/E, so nothing is shown at all.

Screening what you have analysed

Once you have analysed a few dozen companies, the interesting question stops being "what is this one like?" and becomes "which of these meet my conditions?". The Screener page answers that, using queries that read like screener.in's own.

It screens your archive, not the whole market. The universe is every company the app has analysed, taking the newest saved analysis of each. It grows as you use the app — analyse a watchlist on the Research page, or add stocks to a portfolio and let the background refresher work through them. It will never surface a company you have never looked at.

Writing a query

Conditions are a field, a comparison and a value, joined with **AND** and **OR** :

```
ROE > 15 AND P/E < 25
ROCE > 20 AND Sales Growth > 15 OR Sector = Banks
```

AND binds tighter than **OR** , exactly as on screener.in: the second line means "(ROCE above 20 and sales growth above 15) or (anything in Banks)". Queries are read by the app itself, never handed to Python to execute, so a malformed query gives you a plain error message rather than doing anything unexpected.

Field names are forgiving. **ROE** , **roe** and **return on equity** are the same field; so are **P/E** , **pe** and **price to earning** . The **Fields you can use** panel on the right lists every name and the units it expects — get one wrong and the error says which.

Field	Unit	Where it comes from
Market Cap, Current Price, Book Value	₹ Cr / ₹	screener.in's ratios
P/E, Price to Book	ratio	screener.in; price-to-book is worked out from price and book value
ROCE, ROE, OPM, Dividend Yield	%	screener.in; OPM from the latest quarter
Sales Growth, Profit Growth	% year on year	the latest quarter against the same quarter a year earlier
Sector, Verdict	text	screener.in's sector; the app's own verdict
Guidance Score	%	how many of management's graded promises were delivered
Guidance Checked	count	how many promises have actually been graded

Sector and Verdict hold text, so they only accept **=** and **!=** , and they match on part of the word — **Sector = Bank** finds "Banks" and "Private Banks" alike.

Screening on whether management keeps its word

The last two fields exist nowhere else, because they are built from the app's own records. Chapter 10 explains how commitments are pulled out of earnings calls and graded against the results that followed; **Guidance Score** turns that history into a number you can sort on.

```
Guidance Score > 60 AND Guidance Checked >= 3
```

The second half matters. One kept promise is not a track record, so the ready-made **Management keeps its word** screen insists on at least three graded commitments before it will believe a score.

Only graded promises count. A commitment the app has filed but not yet checked is not evidence either way, and one graded *Unclear* — where the reported results simply do not cover it — is not a miss. A company with nothing graded has no score at all, rather than a score of zero, so it is never caught by a query like `Guidance Score < 50`.

Ready-made screens, and your own

The pills on the right fill the box for you: *Quality at a fair price*, *Growing fast*, *Dividend payers*, *Cheap on book*, *AI likes it* and *Management keeps its word*. Change one to taste, then **Save screen** under a name of your own and it joins the list. Saved screens live in the same local database as everything else.

Results show the fields you screened on first, then a few standard columns. Every figure comes from a saved analysis, so a stock you have not re-analysed for a month is screened on month-old numbers — the **Analysed** column tells you which.

Comparing companies side by side

A screen narrows a list. The Compare page is for the two or three names left at the end of it.

Pick between two and four companies from everything you have analysed. If you hold any of them, the page starts with yours already selected. Each gets a card with its name, sector and current verdict, and underneath is a single table of the figures that usually decide it:

market capitalisation, price, P/E, price to book, ROCE, ROE, operating margin, sales growth, profit growth and dividend yield.

Reading the table

One cell on each row is shaded green: the leader on that measure. The app knows which direction is good — **lowest** wins for the valuation multiples (P/E and price to book), **highest** wins for everything else. A row where fewer than two companies have a figure is left unshaded, because there is nothing to lead.

Green means "highest" or "lowest", not "best". The cheapest P/E in a group is often the company the market trusts least, and the highest growth can be one good quarter against a weak one. The shading is there to save you scanning ten rows of numbers, not to pick for you.

The price chart

Below the table, a year of share prices with every company rebased to 100 on the first day, so the lines answer "which of these has done better since last year?" regardless of whether one trades at ₹80 and another at ₹8,000.

Figures come from each company's latest saved analysis, so re-run anything that looks stale before drawing conclusions. Two companies are needed before the page shows anything at all.

Searching everything you have downloaded

Every filing the app downloads stays on your machine and stays searchable — across companies, with no network and no cost per search.

Asking a real question

When a filing is read, its text is stored in a small database alongside the extracted figures. The **Archive** page searches all of it at once. Type a question the way you would ask a colleague — *what did management say about capex and expansion plans?* — and leave **Let the model do the searching and answer** switched on.

Four things then happen, and the model does the two that need judgement:

Step	What happens
1 · Vocabulary	The model lists the words a filing would really use. Ask about "capex" and it adds <i>capital expenditure, expansion plans, growth strategy, investment plans</i> . This is the step plain keyword search cannot do: the gap between your question and a transcript is wording, not logic.
2 · Retrieval	The database finds the pages matching any of those terms and ranks them by relevance.
3 · Judgement	The model reads the shortlist and throws out the coincidences — the page about <i>share capital</i> when you asked about <i>capital expenditure</i> .
4 · Answer	It answers from the passages that survive, quoting the figures and citing each passage as [1], [2] after the claim.

On a real Reliance archive, asking about capex returned an answer citing a capital expenditure of about ₹39,000 crore, drawing on both the concall transcript and the annual report — in six seconds, on a laptop, for about 2,800 tokens. The model only ever sees the passages retrieved from your own documents, so the answer is grounded in the filings rather than in whatever it happens to remember about the company.

Every result shows which company, which filing and which page it came from, along with where that PDF sits on your disk, so you can always go and read the original. You can narrow a search to particular companies or filing types, and expand any result to read the full page.

Switch the toggle off — or run without a model at all — and you still get plain keyword search. It just cannot reason about what it finds.

Ask AI: answers from your own data

The Archive page finds passages. Ask AI answers questions — in plain English, from your portfolio, your saved analyses and your downloaded filings, with every company figure cited back to where it came from.

What it can see

For each question the app gathers up to three kinds of grounding, cheapest first, numbers them, and hands them to the model as sources:

Source	When it is included
Your portfolio	When you ask about "my holdings", "which of my...", or anything without a company name. One row per stock: verdict, price, today's move, P/E, ROE, ROCE and growth.
A saved analysis	For every company you name — its ratios, quarterly figures, strengths, risks and recent headlines, from the most recent run.
Filing passages	The best-matching pages from your downloaded concalls, presentations and annual reports. When your words don't match the filing's — you say "capex", it says "capital expenditure" — the model supplies the wording that would, and searches again.
A whole-archive count	Only for questions asking <i>which</i> companies or <i>how many</i> . Counted across every indexed page rather than the handful read, so the answer is complete.

Why the count matters. The model reads six passages. Ask "which of my companies mentioned China?" when twenty-three of them did, and six passages can only ever produce a confident, wrong "six". The count comes straight from the database, so it is right regardless of what the model reads.

Name no company and ask a follow-up like "and its margins?" and it keeps the companies already under discussion, rather than starting again.

Asking

Type a question, or start from one of the suggestions on the empty screen — they are built around stocks you actually hold. The answer streams as it is written. Underneath it you get:

- **Numbered source chips.** Click any one to see exactly what it was: a portfolio row, an analysis and its date, or a filing with the file on disk and the page number. Every company-specific claim in the answer carries a [2] pointing at one of them.
- **Three follow-up questions**, which come back in the same model call as the answer — so a turn costs one request, not two.
- **A line naming the model**, how long it took and how many sources it used. If your chosen model was busy and another stood in, it says so there.

Keeping it honest

The assistant is told to answer from the sources, cite each company fact, copy figures exactly with their units, and say plainly when the sources do not cover something. It may add general market or accounting knowledge — how a ratio is defined, what a term means — but it has to label that as general knowledge, and it must never invent a company figure.

Check the citation when a number matters. The chips are there to be clicked. An answer with no citation on a company-specific claim is one to be suspicious of, and a smaller local model is more likely to produce one. Chapter 4 has the guidance on model sizes.

Scope and saved chats

The **Scope** button narrows a conversation to chosen companies, and can turn off filing search altogether — useful when you want an answer from the numbers alone, and faster, since no passages are retrieved.

Every conversation is saved and listed in the sidebar under its opening question. Pick one to carry on where you left off, **New chat** to start clean, or delete one you are finished with. Like everything else, they are stored on your own machine.

This page needs a model. With AI switched off, or none set up, it says so and does nothing. A local Ollama model is free and keeps every question on your computer; chapter 4 covers the options.

Excel and Word reports

The **Excel workbook** and **Word report** buttons appear above the tabs after a run. Both contain everything from that run.

Excel workbook

- **Overview sheet:** summary tiles and one row per company, with verdicts, ratios and quarter-on-quarter changes. Click a ticker to jump to its sheet.
- **One sheet per company:** AI analysis, key ratios, strengths and risks, news with links, all quarterly results with a chart, and the filings.
- **Figures sheet:** every number extracted from every PDF in one filterable table. Use the arrows in the header row to filter by ticker, category or label.

Totals and changes are live formulas, so they update if you edit the numbers. Sheets print landscape, one page wide.

Word report

- **Cover page** with an "At a glance" table of all companies.
- **One page per company:** verdict, AI analysis, ratios, strengths and risks, the last 5 quarters, news with clickable links, and the filings.
- A4 layout with page numbers, ready to print or share as PDF (*File → Save As → PDF* in Word).

The Word report shows 5 quarters so the table fits the page. The Excel workbook has all of them.

Finding ticker symbols

The app uses the symbol from the company's page address on screener.in, which is usually its NSE symbol.

1. Go to screener.in and search for the company.
2. Open its page and look at the address bar. The part after `/company/` is the symbol. For `screener.in/company/HDFCBANK/` it's **HDFCBANK**.
3. Type that into the **Tickers** box. Upper or lower case both work.

Company	Symbol	Company	Symbol
Reliance Industries	RELIANCE	HDFC Bank	HDFCBANK
Tata Consultancy Services	TCS	ICICI Bank	ICICIBANK
Infosys	INFY	State Bank of India	SBIN

Some companies listed only on the BSE use a number instead of letters. Whatever appears after `/company/` on screener.in is what the app needs.

Privacy and how the AI is used

The AI runs on your computer through Ollama. None of your data is sent to an AI company.

What leaves your computer

Only ordinary web requests for public pages: company data from screener.in, filing PDFs from the stock exchanges and company websites, and headlines from Google News, Bing News, Economic Times, Livemint, Business Standard and Hindu BusinessLine. The app has no accounts, no tracking and no analytics.

What the AI sees

AI task	Input	Typical tokens
Verdict and analysis	Key ratios, the last two quarters' results, and up to 4 strengths and 4 risks	≈ 400
News digest and sentiment	Up to 8 headlines (not the full articles)	≈ 300
Error explanation	The error message, only when a company fails	≈ 100

A *token* is roughly $\frac{3}{4}$ of a word. The app keeps usage low: it sends only clean, structured data (the thousands of PDF numbers are too noisy); caps every reply's length; switches off hidden "thinking"; and reuses answers for an hour (30 minutes for news), so re-running never pays twice for the same data. The Overview tab shows exactly how many tokens each run used.

Not investment advice. Verdicts come from a small language model reading scraped data. It can misread numbers or miss context. Always check the underlying figures and filings before making decisions.

Using data sources responsibly

The app fetches public pages the way a browser does, pauses between companies, and keeps downloaded PDFs so they're never fetched twice. Please keep it that way: analyse a handful of companies at a time, and respect each website's terms of use.

The Settings page

Almost everything about the app can be changed from inside it. Your choices are stored in `settings.json` next to the app, and nothing here can break it permanently — there is a Reset button at the bottom of the last tab.

Models & keys

Choose your provider, give it an address and a key if it needs one, and pick a model. **Test connection** tells you immediately whether it works and how many models it can see. Under **Generation limits** you can set how long each kind of reply may be, in tokens; these are deliberately tight, and they are the difference between a tidy three-sentence answer and a surprising bill.

Spending

Only relevant if you use a paid provider; a local model is free and is never counted. Here you set a limit per run, a monthly limit in your currency and a monthly token limit, and see exactly what has been spent this month — by provider, by model and by what it was spent on, with a daily chart. A call that would cross a limit is refused before it is sent. Chapter 4 explains the three limits and which one matters most.

Data & scraping

The tickers, filing types and timeframe a new run starts with, plus the limits it works under: the largest filing to download, how many pages to scan, how many filings to fetch at once, how long to pause between companies, and how recent news must be. The same tab has switches to turn whole features off — if you never look at news, or never want anything saved, turn it off here and the tabs disappear.

Setting	Default and meaning
Largest filing	25 MB. Anything bigger is skipped rather than downloaded.
Pages scanned per PDF	150. Most filings are well under this, and a document is only ever scanned once.
Parallel downloads	4. Filings fetch at the same time; higher mostly annoys the servers.
Pause between companies	1–2 seconds. Politeness towards free public sites.
Headline age / count	14 days / 8 headlines.
Result cache	60 minutes. Takes effect after a restart; <i>Clear cached results</i> works at once.

Appearance

Accent colour, text size, spacing, chart colours, table height, decimal places, whether metrics show sparklines, and which tabs appear on the research page and in what order. You can also rewrite the subtitle under the app's title. If anything ever looks wrong, switch off **Apply my appearance settings** and the app falls straight back to its built-in theme.

Performance

How much memory the app is using right now, and every switch that changes it: lean mode, background work, chart preloading, cache sizes and how often held stocks are re-analysed. It is also where you turn on starting the app when you log in. Chapter 20 covers all of it.

Storage & history

This is where you decide how long the app remembers things. Each limit can be set to 0, which means keep forever.

What	Default
Saved analyses	180 days, and at most 40 per company. This is what the History page shows.
Filing PDFs	90 days. Deleting these frees the most space; they are downloaded again when needed.
Searchable text	365 days. Keep this longer than the PDFs — it is small, and it is what the Archive page reads.

With **Apply these limits automatically** switched on, the clear-out happens quietly when the app starts. You can also run it on demand, clear the caches, or — behind a confirmation — delete every saved analysis and everything indexed.

Settings that live outside the app

Two things are still set when you start the app rather than inside it. To run on a different port:

```
python -m streamlit run app.py --server.port 8502
```

And to reach Ollama on another machine you can either type the address into Settings or use the `OLLAMA_HOST` variable, which is the same one Ollama itself reads:

WINDOWS (POWERSHELL)

```
$env:OLLAMA_HOST = "192.168.1.20:11434"
python -m streamlit run app.py
```

MACOS / LINUX

```
OLLAMA_HOST=192.168.1.20:11434 \
python -m streamlit run app.py
```

Keeping it light, and starting it automatically

This app is meant to sit in the background — watching prices, re-analysing what you hold, building a digest once a week. So it is built to be something you can forget about: comfortably under 500 MB of memory, and entirely under your control.

What it actually uses

Settings → **Performance** shows the memory this session is using, right now, and warns you if it goes past 500 MB. For reference, with every page opened it sits around 230 MB, and about 190 MB in lean mode. Most of that is Python itself plus the two libraries the app is built on; the app's own data is a small fraction.

The switches

Setting	What it trades
Lean mode	One switch for a light footprint: no chart preloading, small caches. The first chart you open takes a few seconds longer.
Keep working in the background	Off, the app does nothing unless you open a page and ask. No re-analysis, no price alerts, no weekly digest.
Preload the charting library	About 30 MB to make the first chart instant instead of taking a few seconds.
Price histories	The largest thing the app holds in memory — years of daily bars per company. Lower means less memory and more re-fetching.
Latest quotes / Company searches	How many results are remembered before the app asks again.
Let Ask AI rank passages	The model picks which retrieved passages actually answer your question, instead of trusting the keyword order — "share capital" is not capital expenditure. One extra request per question: a few seconds locally, longer on a hosted provider.
Re-analyse each held stock every...	How often background analysis runs. 0 analyses a stock once, when you add it, and never again.
Filings downloaded at once	Faster, but heavier — and less polite to the site being read.

The cache sizes take effect when the app restarts. **Free memory now** drops everything held in memory immediately; nothing is lost, it is simply fetched or recomputed next time it is needed.

The app is already careful about the expensive things. A filing is parsed once ever and the result cached by file hash. The PDF reader — about 22 MB — is only loaded when a PDF is actually opened, so a session that never runs an analysis never pays for it. Model answers are cached, so re-running a company costs nothing.

Starting when you log in

At the bottom of the same tab, **Start when I log in** makes the app launch quietly in the background whenever you sign in to your computer, so alerts and the weekly digest keep arriving without you starting anything.

Your system	What gets written
macOS	A LaunchAgent in <code>~/Library/LaunchAgents</code> , at background priority so it never steals focus, and restarted if it ever crashes.
Windows	A small script in your Startup folder that launches the app minimised.
Linux	A <code>.desktop</code> entry in <code>~/.config/autostart</code> .

It is exactly one file, in the place your operating system documents for it, and switching the toggle off deletes it again. There is no installer and no background service left behind. Once it is running, open <http://localhost:8501> in a browser whenever you want to look at it.

It uses the same Python you installed the app with, so a virtual environment keeps working after a reboot. If you move the app folder, switch the toggle off and on again so it points at the new location.

Troubleshooting

Find what you see in the left column.

Installing and starting

What you see	What to do
<code>python</code> or <code>python3</code> "not found"	Install Python from python.org (chapter 3, step 1). On Windows, re-run the installer and tick <i>Add python.exe to PATH</i> , then open a new PowerShell window.
"running scripts is disabled on this system" (Windows)	Don't activate the environment; prefix commands with <code>.venv\Scripts\</code> instead, e.g. <code>.venv\Scripts\python -m streamlit run app.py</code> .
Errors during <code>pip install</code>	Check <code>python --version</code> is 3.10 or newer, then upgrade pip and retry: <code>python -m pip install --upgrade pip</code> .
"No module named streamlit"	The environment isn't active. Run the activate command from chapter 3, step 3, then try again.
"Port 8501 is already in use"	The app is already running in another window, so use that one. Or start it on another port: <code>--server.port 8502</code> .

Running an analysis

What you see	What to do
A company shows Failed	Usually a wrong symbol: check it on screener.in (chapter 17). It can also be a network problem or the site temporarily limiting requests; wait a few minutes and try again. With AI on, the card explains the problem in plain English.
A filing says <i>Skipped</i>	It was over 25 MB, or the link led to a web page rather than a PDF. The rest of the analysis is unaffected.
"No news in the last 14 days"	Normal for smaller companies. The news sources simply had nothing that named the company.
Numbers look out of date	Results are reused for an hour. Use : → Clear cache , then run again.

Local AI

What you see	What to do
Sidebar says Ollama not detected	Start Ollama (open the app, or run <code>ollama serve</code>), then refresh the page. Check localhost:11434 says <i>Ollama is running</i> .
Connected, but the model list is empty	No model has been downloaded yet. Run <code>ollama pull <name></code> with a model from ollama.com/library that fits your RAM (chapter 4, route A), then press Test connection again.
"... returned no summary for this ticker"	Update Ollama to the latest version (0.9 or newer is required), or pick another model from the list.
The first AI answer is slow	The model is loading into memory, which can take 10 to 30 seconds. Later answers take a second or two.
Computer slows down	The model is too large for your RAM. Choose a smaller one. <code>ollama list</code> shows installed models; <code>ollama rm <name></code> removes one.

Reports

What you see	What to do
Excel or Word warns about the file	Please report it (see the project's GitHub <i>Issues</i> page) with the tickers you used. Both files are checked automatically before download, so this shouldn't happen.
Totals show 0 in a file previewer	Some previewers (e.g. macOS Quick Look) don't calculate formulas. Open the file in Excel, Numbers, LibreOffice or Google Sheets.

Glossary

The terms you'll meet in the app, in plain language.

Ticker / symbol

A company's short code on the stock exchange, e.g. INFY for Infosys.

Crore (Cr)

10 million (1,00,00,000). Indian financial figures are usually in ₹ crore.

Market cap

Share price × number of shares: what the whole company is worth on the market.

P/E (price-to-earnings)

Share price ÷ earnings per share. Higher means investors pay more for each rupee of profit.

Book value

The company's net assets per share, from its accounts.

ROE (return on equity)

Profit as a percentage of shareholders' money. Shows how efficiently that money is used.

ROCE (return on capital employed)

Operating profit as a percentage of all capital used, including debt.

Dividend yield

Yearly dividend as a percentage of the share price.

Face value

The nominal value printed on a share. Rarely matters for analysis.

Sales / revenue

Money earned from the company's business in the period.

Operating profit

Sales minus operating expenses, before interest, depreciation and tax.

OPM (operating profit margin)

Operating profit as a percentage of sales.

Net profit

What's left after all expenses, interest and tax.

EPS (earnings per share)

Net profit ÷ number of shares, in rupees.

QoQ (quarter on quarter)

Change from the previous quarter. Rising sales or profit is shown in green; rising costs are shown in red.

pp (percentage points)

The change between two percentages: a margin going from 20% to 23% is +3 pp.

Concall transcript

The written record of management's earnings call with analysts.

Investor presentation

The slide deck a company publishes alongside its results.

Ollama

Free software that runs AI language models on your own computer.

Model

The AI "brain" Ollama runs. Bigger models are usually more accurate but need more memory.

Token

A chunk of text, roughly ¾ of a word, that AI models read and write.

Verdict

The AI's one-word take (Positive, Neutral or Cautious), based only on the scraped data.